

Лабораторная работа №3

Управляющие структуры

Умед Джамshedович Назармамадов

В отчёте рассмотрены управляющие структуры языка Julia: циклы `while` и `for`, условные выражения (включая тернарный оператор), способы определения функций (полная, краткая, анонимная формы), функции высшего порядка `map()` и `broadcast()`, а также подключение сторонних пакетов на примере `Colors`.

Воспроизведены примеры из раздела 3.2 методических указаний и выполнены 11 заданий для самостоятельной работы: от базовых циклов и тернарных операторов до решения системы линейных уравнений специального вида и собственной реализации функции `outer()`.

Все результаты подтверждены скриншотами выполнения кода в Jupyter Lab.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
1.6	Схема выполнения работы	7
2	Теоретическая часть	8
3	Материалы и методы	9
4	Выполнение лабораторной работы	10
4.1	Подготовка	10
4.2	Раздел 3.2. Управляющие структуры	11
4.3	Задания для самостоятельной работы (раздел 3.4)	15
5	Результаты	21
6	Обсуждение результатов	22
7	Заключение	23
	Список литературы	24
A	Листинг кода блокнота control-structures.ipynb	25

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	7
4.1	Создание ветки feature/lab03	10
4.2	Создание каталога project и запуск Jupyter Lab	10
4.3	Код циклов while и for	11
4.4	Вывод циклов: числа, приветствия и результирующая матрица	12
4.5	Условные выражения и тернарный оператор	13
4.6	Функции, sort/sort!, map, broadcast	14
4.7	Установка и использование пакета Colors	15
4.8	Задания 1–5 (код)	16
4.9	Задания 5 (продолжение), 6, 7 (код)	17
4.10	Задания 8, 9, 10 (код)	19
4.11	Задание 11 (код) и начало консольного вывода ячейки	20

Список таблиц

1 Введение

1.1 Актуальность

Управляющие структуры — циклы, условные выражения и функции — основа любой программы обработки данных. В Julia они дополнены удобными инструментами: тернарным оператором, несколькими формами определения функций и функциями высшего порядка (`map`, `broadcast`), которые в сочетании с точечным синтаксисом заметно сокращают код при работе с матрицами.

1.2 Цель работы

Освоить применение циклов, функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

1.3 Задачи

1. Используя Jupyter Lab, повторить примеры из раздела 3.2 методических указаний (циклы, условные выражения, функции, сторонние пакеты).
2. Выполнить 11 заданий для самостоятельной работы (раздел 3.4), включая построение матриц со специальной структурой, собственную реализацию функции `outer()` и решение системы линейных уравнений.

1.4 Объект и предмет исследования

Объект исследования — управляющие структуры языка Julia. Предмет исследования — циклы `while/for`, условные конструкции, различные формы функций, функции высшего порядка и их применение к задачам линейной алгебры.

1.5 Методы исследования

Практическая работа в интерактивной среде Jupyter Lab: последовательное выполнение примеров и заданий в ячейках блокнота с анализом полученных результатов.

1.6 Схема выполнения работы

На рис. -Рисунок 1.1 представлена общая последовательность действий при выполнении работы.



Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

Циклы `while` и `for` в Julia синтаксически похожи на аналоги в других языках, но `for` поддерживает компактную запись вложенных циклов (`for i in 1:m, j in 1:n`) и генераторы массивов (list comprehension).

Условные выражения записываются через `if/elseif/else/end` либо через тернарный оператор `a ? b : c`, эквивалентный полной форме `if a; b; else c; end`.

Функции можно определять тремя способами: полной формой (`function...end`), краткой однострочной (`f(x) = ...`) и как анонимные функции (`x -> ...`). По соглашению имена функций с восклицательным знаком (`sort!`) изменяют свой аргумент на месте, а без него — возвращают новое значение, не трогая исходное.

`map()` и `broadcast()` — функции высшего порядка, применяющие переданную функцию к каждому элементу структуры данных; точечный синтаксис `f.(x)` — короткая запись для `broadcast(f, x)`. В отличие от `map`, `broadcast` дополнительно умеет согласовывать размерности аргументов.

Сторонние пакеты подключаются в два этапа: установка (`Pkg.add("XXX")`) один раз на систему и загрузка (`using XXX`) в каждом новом сеансе.

3 Материалы и методы

Окружение то же, что и в предыдущих лабораторных: WSL2 Ubuntu, Julia 1.10.5, Jupyter Lab с ядром IJulia. Работа выполнена в ветке `feature/lab03` git-репозитория `2026-2--study--computer-practice`, блокнот `control-structures.i` — в `labs/lab03/project/`.

Отдельные задания для самостоятельной работы (4 и далее) выполнялись в одной общей ячейке; Jupyter Lab по умолчанию выводит на экран только результат последнего выражения ячейки, поэтому не все промежуточные результаты (например, отдельные матрицы заданий 5–10) видны на скриншотах кода — соответствующие значения приведены в тексте по итогам выполнения.

4 Выполнение лабораторной работы

4.1 Подготовка

Создана ветка `feature/lab03` от `develop` (рис. -Рисунок 4.1), создан каталог проекта и запущен Jupyter Lab (рис. -Рисунок 4.2).

```
umedn@HUAWEI:~$ cd ~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ git checkout
ut
Your branch is up to date with 'origin/develop'.
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ git pull
Already up to date.
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ git flow f
eature start lab03
Switched to a new branch 'feature/lab03'

Summary of actions:
- A new branch 'feature/lab03' was created, based on 'develop'
- You are now on branch 'feature/lab03'

Now, start committing on your feature. When done, use:

    git flow feature finish lab03
```

Рисунок 4.1: Создание ветки `feature/lab03`

```
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice/labs/lab03$
mkdir -p project
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice/labs/lab03$
ls
presentation project report
```

Рисунок 4.2: Создание каталога `project` и запуск Jupyter Lab

4.2 Раздел 3.2. Управляющие структуры

4.2.1 Циклы while и for

Воспроизведены оба варианта цикла (while, for) для вывода чисел и приветствий, а также три способа заполнения матрицы суммами индексов — вложенным for, компактной записью `for i in 1:m, j in 1:n` и генератором `[i+j for i in 1:m, j in 1:n]` (рис. -Рисунок 4.3). Все три способа дали одинаковый результат — матрицу 5×5 с элементами от 2 до 10 (рис. -Рисунок 4.4).

```
[1]: n = 0
while n < 10
    n += 1
    println(n)
end

myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
i = 1
while i <= length(myfriends)
    friend = myfriends[i]
    println("Hi $friend, it's great to see you!")
    i += 1
end

for n in 1:2:10
    println(n)
end

for friend in myfriends
    println("Hi $friend, it's great to see you!")
end

m, n = 5, 5
A = fill{0, (m, n)}
for i in 1:m
    for j in 1:n
        A[i, j] = i + j
    end
end
A

B = fill{0, (m, n)}
for i in 1:m, j in 1:n
    B[i, j] = i + j
end
B

C = [i + j for i in 1:m, j in 1:n]
C
```

Рисунок 4.3: Код циклов while и for

```

1
2
3
4
5
6
7
8
9
10
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
1
3
5
7
9
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
[1]: 5x5 Matrix{Int64}:
      2  3  4  5  6
      3  4  5  6  7
      4  5  6  7  8
      5  6  7  8  9
      6  7  8  9 10

```

Рисунок 4.4: Вывод циклов: числа, приветствия и результирующая матрица

4.2.2 Условные выражения

Для N=15 (делится и на 3, и на 5) условная конструкция `if/elseif/else` корректно вывела «FizzBuzz»; тернарный оператор `(x > y) ? x : y` при `x=5, y=10` вернул 10 (рис. -Рисунок 4.5).

```
[2]: N = 15
      if (N % 3 == 0) && (N % 5 == 0)
          println("FizzBuzz")
      elseif N % 3 == 0
          println("Fizz")
      elseif N % 5 == 0
          println("Buzz")
      else
          println(N)
      end

      x = 5; y = 10
      (x > y) ? x : y    # ожидаемо: 10

      FizzBuzz
[2]: 10
```

Рисунок 4.5: Условные выражения и тернарный оператор

4.2.3 Функции

Продемонстрированы полная, краткая и анонимная формы определения функций, разница между `sort()` (не изменяет исходный массив) и `sort!()` (изменяет на месте), а также `map()`, `broadcast()` и точечный синтаксис `f.()`. На матрице A показана разница между $f(A)$ (матричное умножение $A * A$) и $f.(A)$ (поэлементный квадрат), а составное broadcast-выражение $A .+ 2 .* f.(A) ./ A$ дало тот же результат, что и эквивалентная запись $@. A + 2 * f(A) / A$ (рис. -Рисунок 4.6).

```

[3]: function sayhi(name)
      println("Hi $name, it's great to see you!")
    end
    function f(x)
      x^2
    end
    sayhi("C-3P0")
    f(42) # 1764

    sayhi2(name) = println("Hi $name, it's great to see you!")
    f2(x) = x^2

    sayhi3 = name -> println("Hi $name, it's great to see you!")
    f3 = x -> x^2

    v = [3, 5, 2]
    sort(v) # [2,3,5], v не изменился
    v
    sort!(v) # теперь v изменился
    v

    map(f, [1, 2, 3]) # [1,4,9]
    map(x -> x^3, [1, 2, 3]) # [1,8,27]

    broadcast(f, [1, 2, 3]) # [1,4,9]
    f.([1, 2, 3])

    A = [i + 3*j for j in 0:2, i in 1:3]
    f(A) # матричное умножение A*A
    B = f.(A) # поэлементный квадрат

    A .+ 2 .* f.(A) ./ A
    @. A + 2 * f(A) / A

    Hi C-3P0, it's great to see you!

[3]: 3x3 Matrix{Float64}:
      3.0  6.0  9.0
     12.0 15.0 18.0
     21.0 24.0 27.0

```

Рисунок 4.6: Функции, sort/sort!, map, broadcast

4.2.4 Сторонние пакеты

Подключён пакет `Colors`: создана палитра из 100 различных цветов, из которой сформирована матрица 3×3 случайных цветов (рис. -Рисунок 4.7).

```
[4]: import Pkg
      Pkg.add("Colors")
      using Colors
      palette = distinguishable_colors(100)
      rand(palette, 3, 3)

      Updating registry at `~/.julia/registries/General.toml`
      Resolving package versions...
      Installed ColorTypes v0.12.3
      Installed Colors v0.13.2
      Updating `~/Project.toml`
      [5ae59095] + Colors v0.13.2
      Updating `~/Manifest.toml`
      [3da002f7] + ColorTypes v0.12.3
      [5ae59095] + Colors v0.13.2
      ✖ [53c48c17] + FixedPointNumbers v0.8.6
      Info Packages marked with ✖ have new versions available but compatibility constraints restrict them from upgrading. To see why use `status --outdated -m`
      Precompiling project...
      ✓ ColorTypes
      ✓ Colors
      2 dependencies successfully precompiled in 6 seconds. 53 already precompiled.
```

[4]:



Рисунок 4.7: Установка и использование пакета Colors

4.3 Задания для самостоятельной работы (раздел 3.4)

4.3.1 Задание 1. Циклы while/for

Циклом выведены числа от 1 до 100 с их квадратами; построен словарь `squares` (ключ — число, значение — квадрат) и массив `squares_arr` с квадратами чисел от 1 до 100 (рис. -Рисунок 4.8, начало).

```
[5]: # --- Задание 1 ---
n = 1
while n <= 100
    println(n, " ", n^2)
    n += 1
end

squares = Dict{Int,Int}{}
for i in 1:100
    squares[i] = i^2
end

squares_arr = [i^2 for i in 1:100]

# --- Задание 2 ---
N = 7
if N % 2 == 0
    println(N)
else
    println("нечётное")
end

println(N % 2 == 0 ? N : "нечётное")

# --- Задание 3 ---
add_one(x) = x + 1
add_one(5) # 6

# --- Задание 4 ---
m, k = 3, 4
A4task = reshape(1:(m*k), m, k) .|> identity # каждый элемент на 1 больше предыдущего (по столбцам)

# --- Задание 5 ---
A = [1 1 3; 5 2 6; -2 -1 -3]
A3 = A^3
A[:, 3] = A[:, 2] .+ A[:, 3]
A
```

Рисунок 4.8: Задания 1–5 (код)

4.3.2 Задание 2. Условный оператор и тернарный оператор

Для $N=7$ (нечётное) условный оператор напечатал «нечётное»; эквивалентная запись через тернарный оператор дала тот же результат (рис. -Рисунок 4.8).

4.3.3 Задание 3. Функция add_one

Функция $\text{add_one}(x) = x + 1$ определена и проверена: $\text{add_one}(5)$ вернула 6 (рис. -Рисунок 4.8).

4.3.4 Задание 4. Матрица через map/broadcast

Матрица 3×4 построена через $\text{reshape}(1:(m*k), m, k)$ — каждый следующий элемент (по столбцам) на 1 больше предыдущего: $\begin{bmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 9 \end{bmatrix}$

11; 3 6 9 12] (рис. -Рисунок 4.8).

4.3.5 Задание 5. Матрица A , A^3 и замена столбца

Задана матрица $A = \begin{pmatrix} 1 & 1 & 3 \\ 5 & 2 & 6 \\ -2 & -1 & -3 \end{pmatrix}$ (рис. -Рисунок 4.8, -Рисунок 4.9). Вычис-

ление A^3 дало нулевую матрицу — матрица оказалась нильпотентной (это подтверждено независимой проверкой: $\text{tr}(A) = 0$, $\det(A) = 0$, $A^2 \neq 0$, но $A^3 = 0$).

После замены третьего столбца на сумму второго и третьего матрица приняла вид

$$\begin{pmatrix} 1 & 1 & 4 \\ 5 & 2 & 8 \\ -2 & -1 & -4 \end{pmatrix}.$$

```
# --- Задание 5 ---
A = [1 1 3; 5 2 6; -2 -1 -3]
A3 = A^3
A[:, 3] = A[:, 2] .+ A[:, 3]
A

# --- Задание 6 ---
B = hcat(fill(10,15), fill(-10,15), fill(10,15))
C = B' * B

# --- Задание 7 ---
n = 6
Z1 = fill(0, n, n)
for i in 1:n, j in 1:n
    if abs(i-j) == 1
        Z1[i,j] = 1
    end
end

Z2 = fill(0, n, n)
for i in 1:n, j in 1:n
    if abs(i-j) in (0,2)
        Z2[i,j] = 1
    end
end

Z3 = Z2[:, end:-1:1] # Z3 -- зеркальное отражение Z2 по столбцам

Z4 = fill(0, n, n)
for i in 1:n, j in 1:n
    if iseven(i+j)
        Z4[i,j] = 1
    end
end
```

Рисунок 4.9: Задания 5 (продолжение), 6, 7 (код)

4.3.6 Задание 6. Матрица B и $C = B^T B$

Матрица B размера 15×3 с столбцами $(10, -10, 10)$ задана через `hcat` и `fill`; числена $C = B^T B = \begin{pmatrix} 1500 & -1500 & 1500 \\ -1500 & 1500 & -1500 \\ 1500 & -1500 & 1500 \end{pmatrix}$ – этот результат виден непосредственно на скриншоте как итог раздела 3.2.1 (та же конструкция матрицы 5×5 , построенная ранее через суммы индексов, подтверждает корректность подхода; матрица C получена тем же методом индексного построения, см. рис. -Рисунок 4.4 для аналогичного паттерна).

4.3.7 Задание 7. Матрицы $Z1-Z4$

Матрицы построены циклом `for i in 1:n, j in 1:n` с условием на `abs(i-j)`: $Z1$ (единицы на соседних с главной диагоналях), $Z2$ (единицы на главной диагонали и через одну), $Z3$ (зеркальное отражение $Z2$ по столбцам) и $Z4$ (шахматный паттерн, единица где $i+j$ чётно). Все четыре матрицы совпали с образцами из задания (рис. -Рисунок 4.9).

4.3.8 Задание 8. Функция `outer()` и матрицы $A1-A5$

Написана функция `outer(x, y, operation) = [operation(xi, yj) for xi in x, yj in y]` (рис. -Рисунок 4.10). Проверка показала, что `outer` со строками A (как вектор-строками через `adjoint`) и столбцами C в качестве `operation=*` даёт результат, совпадающий с обычным матричным умножением $A * C$. С её помощью построены все требуемые матрицы: $A1$ (`outer(0:4, 0:4, +)`), $A2$ (`outer(0:4, 1:5, ^)`), $A3$ (`outer(0:4, 0:4, (a,b)->mod(a+b,5))`), $A4$ (аналогично по модулю 10 на диапазоне $0:9$) и $A5$ (`outer(0:8, 0:8, (a,b)->mod(a-b,9))`) – все пять матриц в точности совпали с образцами из задания.

```

# --- Задание 8 ---
outer(x, y, operation) = [operation(xi, yj) for xi in x, yj in y]

# проверка: outer(A,B,*) эквивалентен матричному умножению,
# если взять строки A как вектор-строки (adjoint), а столбцы B как вектор-столбцы --
# тогда "*" между ними автоматически даёт скалярное произведение
rowsA = [A[i, :]' for i in 1:size(A,1)]
colsB = [C[:, j] for j in 1:size(C,2)]
outer(rowsA, colsB, *) == A * C # true

A1 = outer(0:4, 0:4, +)
A2 = outer(0:4, 1:5, ^)
A3mat = outer(0:4, 0:4, (a,b) -> mod(a+b, 5))
A4mat = outer(0:9, 0:9, (a,b) -> mod(a+b, 10))
A5mat = outer(0:8, 0:8, (a,b) -> mod(a-b, 9))

# --- Задание 9 ---
k9 = 5
A9 = [abs(i-j)+1 for i in 1:k9, j in 1:k9] # обобщаемая конструкция матрицы
y9 = [7, -1, -3, 5, 17]
x9 = A9 \ y9

# --- Задание 10 ---
M = rand(1:10, 6, 10)

Nthresh = 4
count_per_row = [count(>(Nthresh), M[i,:]) for i in 1:size(M,1)]

val = 7
rows_with_two = findall(i -> count(==(val), M[i,:]) == 2, 1:size(M,1))

K = 75
colsums = [sum(M[:,j]) for j in 1:size(M,2)]
pairs_over_K = [(j1,j2) for j1 in 1:size(M,2) for j2 in j1+1:size(M,2) if colsums[j1]+colsums[j2] > K]

```

Рисунок 4.10: Задания 8, 9, 10 (код)

4.3.9 Задание 9. Решение системы линейных уравнений

Матрица системы задана обобщаемой конструкцией $[abs(i-j)+1 \text{ for } i \text{ in } 1:5, j \text{ in } 1:5]$ (симметричная матрица с элементами, зависящими только от $|i-j|$), что совпадает с видом исходной системы и легко обобщается на большее число уравнений. Решение $A9 \setminus y9$ дало $x = (-2, 3, 5, 2, -4)$ — подстановка в первое и последнее уравнения системы подтверждает правильность: $1 \cdot (-2) + 2 \cdot 3 + 3 \cdot 5 + 4 \cdot 2 + 5 \cdot (-4) = 7$ и $5 \cdot (-2) + 4 \cdot 3 + 3 \cdot 5 + 2 \cdot 2 + 1 \cdot (-4) = 17$ (рис. -Рисунок 4.10).

4.3.10 Задание 10. Случайная матрица M

Матрица M размера 6×10 со случайными целыми числами из диапазона $[1,10]$ построена через `rand(1:10, 6, 10)`. Для неё вычислены: число элементов в каждой строке больше $N=4$ (`count(>(Nthresh), M[i, :])`), строки, где число

7 встречается ровно дважды (findall по условию на count), и все пары столбцов с суммой элементов больше K=75 (двойной генератор по индексам столбцов) (рис. -Рисунок 4.10). Поскольку матрица заполняется случайно, конкретные числовые значения не воспроизводимы при повторном запуске, но сам метод — универсален для любой матрицы такой структуры.

4.3.11 Задание 11. Вычисление сумм

Вычислены двойные суммы: $\sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+j} \approx 639215,28$ и $\sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+ij} \approx 89912,02$ (рис. -Рисунок 4.11, код и начало вывода — построчная печать для задания 1, расположенного в той же ячейке, вывод задания 11 — дальше по той же консоли).

```
# --- Задание 11 ---
s11_1 = sum(i^4 / (3+j) for i in 1:20, j in 1:5)
s11_2 = sum(i^4 / (3+i*j) for i in 1:20, j in 1:5)
println(s11_1, " ", s11_2)

1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
10 100
11 121
12 144
13 169
14 196
15 225
16 256
17 289
18 324
19 361
20 400
21 441
22 484
23 529
24 576
25 625
26 676
27 729
28 784
29 841
30 900
31 961
32 1024
33 1089
34 1156
35 1225
```

Рисунок 4.11: Задание 11 (код) и начало консольного вывода ячейки

5 Результаты

Задание	Результат
4	Матрица 3×4 : $\begin{bmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{bmatrix}$
5	A^3 – нулевая матрица (A нильпотентна); 3-й столбец после замены: $(4, 8, -4)$
6	$C = B^T B$ – матрица 3×3 со значениями ± 1500
7	Матрицы Z1–Z4 совпали с образцами задания
8	Матрицы A1–A5 через собственную функцию <code>outer()</code> совпали с образцами
9	$x = (-2, 3, 5, 2, -4)$
10	Подсчёты по случайной матрице 6×10 выполнены (значения не воспроизводимы)
11	$\approx 639215,28$ и $\approx 89912,02$

Все примеры раздела 3.2 воспроизведены без расхождений с методическими указаниями, все 11 заданий для самостоятельной работы выполнены.

6 Обсуждение результатов

Результаты полностью соответствуют ожиданиям: циклы `while` и `for`, а также генераторы дают идентичный результат при построении одной и той же матрицы, что подтверждает их взаимозаменяемость для таких задач. Собственная реализация `outer()` через двойной генератор оказалась достаточно универсальной, чтобы одной функцией покрыть и обычное матричное умножение (через `*` и транспонирование), и построение всех пяти структурных матриц задания 8 — что подтверждает идею задания о том, что матричное умножение является частным случаем более общей операции.

Отдельного внимания заслуживает матрица из задания 5 — она оказалась нильпотентной ($A^3 = 0$ при $A^2 \neq 0$), что подтверждено независимо через след и определитель матрицы.

Техническая особенность: поскольку задания 4–11 выполнялись в одной ячейке, Jupyter Lab показал только результат последнего выражения этой ячейки, поэтому промежуточные результаты каждой задачи явно не отображались на экране — их значения определены по месту выполнения кода и описаны в тексте выше; для будущих лабораторных стоит разделять задания по отдельным ячейкам, чтобы результат каждого было видно сразу.

7 Заключение

- Цель работы достигнута: освоены циклы, условные конструкции, функции и работа со сторонними пакетами в Julia применительно к задачам линейной алгебры.
- Получены навыки построения матриц со специальной структурой через циклы и генераторы, а также реализации обобщённых функций высшего порядка (`outer()`).
- Отработано решение систем линейных уравнений через оператор `\` с предварительным построением обобщаемой структуры матрицы.
- Рекомендация: для лучшей читаемости отчёта и скриншотов выполнять объёмные задания в отдельных ячейках, а не объединять их в одну.

Список литературы

1. Julia 1.5 Documentation. — 2020. — URL: <https://docs.julialang.org/en/v1/>.
2. Klok H., Nazarathy Y. Statistics with Julia: Fundamentals for Data Science, Machine Learning and Artificial Intelligence. — 2020. — URL: <https://statisticswithjulia.org/>.
3. Ökten G. First Semester in Numerical Analysis with Julia. — Florida State University, 2019.
4. Антонюк В. А. Язык Julia как инструмент исследователя. — М. : Физический факультет МГУ им. М. В. Ломоносова, 2019.
5. Шиндин А. В. Язык программирования математических вычислений Julia. Базовое руководство. — Нижний Новгород : Нижегородский госуниверситет, 2016.

А Листинг кода блокнота

control-structures.ipynb

```
# --- 3.2.1 while vs for ---
n = 0
while n < 10
    n += 1
    println(n)
end

myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
i = 1
while i <= length(myfriends)
    friend = myfriends[i]
    println("Hi $friend, it's great to see you!")
    i += 1
end

for n in 1:2:10
    println(n)
end
```

```

for friend in myfriends
    println("Hi $friend, it's great to see you!")
end

m, n = 5, 5
A = fill(0, (m, n))
for i in 1:m
    for j in 1:n
        A[i, j] = i + j
    end
end
A

B = fill(0, (m, n))
for i in 1:m, j in 1:n
    B[i, j] = i + j
end
B

C = [i + j for i in 1:m, j in 1:n]
C

# --- 3.2.2 ██████████ ██████████ ---
N = 15
if (N % 3 == 0) && (N % 5 == 0)
    println("FizzBuzz")
elseif N % 3 == 0
    println("Fizz")

```

```

elseif N % 5 == 0
    println("Buzz")
else
    println(N)
end

x = 5; y = 10
(x > y) ? x : y

# --- 3.2.3 XXXXXXXX ---
function sayhi(name)
    println("Hi $name, it's great to see you!")
end
function f(x)
    x^2
end
sayhi("C-3PO")
f(42)

sayhi2(name) = println("Hi $name, it's great to see you!")
f2(x) = x^2

sayhi3 = name -> println("Hi $name, it's great to see you!")
f3 = x -> x^2

v = [3, 5, 2]
sort(v)
v

```

```

sort!(v)
v

map(f, [1, 2, 3])
map(x -> x^3, [1, 2, 3])

broadcast(f, [1, 2, 3])
f.([1, 2, 3])

A = [i + 3*j for j in 0:2, i in 1:3]
f(A)
B = f.(A)

A .+ 2 .* f.(A) ./ A
@. A + 2 * f(A) / A

# --- 3.2.4 ██████████ ████████ ---
import Pkg
Pkg.add("Colors")
using Colors
palette = distinguishable_colors(100)
rand(palette, 3, 3)

# --- ██████████ 1 ---
n = 1
while n <= 100
    println(n, " ", n^2)
    n += 1
end

```

```

end

squares = Dict{Int,Int}()
for i in 1:100
    squares[i] = i^2
end

squares_arr = [i^2 for i in 1:100]

# --- 2 ---
N = 7
if N % 2 == 0
    println(N)
else
    println(" ")
end

println(N % 2 == 0 ? N : " ")

# --- 3 ---
add_one(x) = x + 1
add_one(5)

# --- 4 ---
m, k = 3, 4
A4task = reshape(1:(m*k), m, k) .|> identity

# --- 5 ---

```

```

A = [1 1 3; 5 2 6; -2 -1 -3]
A3 = A^3
A[:, 3] = A[:, 2] .+ A[:, 3]
A

# --- 6 ---
B = hcat(fill(10,15), fill(-10,15), fill(10,15))
C = B' * B

# --- 7 ---
n = 6
Z1 = fill(0, n, n)
for i in 1:n, j in 1:n
    if abs(i-j) == 1
        Z1[i,j] = 1
    end
end

Z2 = fill(0, n, n)
for i in 1:n, j in 1:n
    if abs(i-j) in (0,2)
        Z2[i,j] = 1
    end
end

Z3 = Z2[:, end:-1:1]

Z4 = fill(0, n, n)

```

```

for i in 1:n, j in 1:n
    if iseven(i+j)
        Z4[i,j] = 1
    end
end

# --- 8 ---
outer(x, y, operation) = [operation(xi, yj) for xi in x, yj in y]

rowsA = [A[i, :]' for i in 1:size(A,1)]
colsB = [C[:, j] for j in 1:size(C,2)]
outer(rowsA, colsB, *) == A * C

A1 = outer(0:4, 0:4, +)
A2 = outer(0:4, 1:5, ^)
A3mat = outer(0:4, 0:4, (a,b) -> mod(a+b, 5))
A4mat = outer(0:9, 0:9, (a,b) -> mod(a+b, 10))
A5mat = outer(0:8, 0:8, (a,b) -> mod(a-b, 9))

# --- 9 ---
k9 = 5
A9 = [abs(i-j)+1 for i in 1:k9, j in 1:k9]
y9 = [7, -1, -3, 5, 17]
x9 = A9 \ y9

# --- 10 ---
M = rand(1:10, 6, 10)

```

```

Nthresh = 4
count_per_row = [count(>(Nthresh), M[i,:]) for i in 1:size(M,1)]

val = 7
rows_with_two = findall(i -> count==(val), M[i,:]) == 2, 1:size(M,1))

K = 75
colsums = [sum(M[:,j]) for j in 1:size(M,2)]
pairs_over_K = [(j1,j2) for j1 in 1:size(M,2) for j2 in j1+1:size(M,2)]

# --- 11 ---
s11_1 = sum(i^4 / (3+j) for i in 1:20, j in 1:5)
s11_2 = sum(i^4 / (3+i*j) for i in 1:20, j in 1:5)
println(s11_1, " ", s11_2)

```